부록. 로컬 LLM 가이드

- 로컬 LLM 설정 방법
  - Ollama 설치 및 모델 다운로드
  - Hermes Agent에 Ollama 연동
  - LM Studio 정식 어댑터
  - 컨텍스트 길이 설정
  - LAN 환경에서의 연동
  - 연동 확인
- 실제 테스트 비교
- 핵심 발견
- VRAM별 모델 추천
- 에이전트용 유망 로컬 모델 (2026년 기준)
- 로컬 모델을 판정 모델로 활용하기
- 외부 메모리 백엔드(Honcho)에 로컬 LLM 연결하기
  - Honcho의 .env 설정 (Ollama 연결)
  - 모델 선택 시 주의할 점
  - .env 변경 후 반영
- 에이전트의 토큰 오버헤드
  - 비용에 미치는 영향
  - 컨텍스트 한계와 워크플로 선택

이 부록은 Ollama·LM Studio 같은 **로컬 LLM 서버를 Hermes Agent에 붙이는 방법**, 모델 비교 테스트 결과, 그리고 판정 모델이나 외부 메모리 백엔드 같은 **로컬 모델 응용**을 다룹니다. 클라우드 API 연동은 2-3절에서 다룹니다.

로컬 LLM 설정 방법

로컬 LLM을 사용하면 데이터가 외부로 나가지 않아 완전한 데이터 주권을 확보할 수 있습니다. 가장 흔한 두 가지 옵션은 **Ollama**(가벼운 명령줄 서버)와 **LM Studio**(GUI 데스크톱 앱)입니다.

> ⚠️ ***한국어 환경의 현실적 한계**: 2026년 3월 기준 테스트에서 Qwen3 14B, Qwen 2.5 14B 등은 한국어 맥락 파악이나 도구 호출 정확도가 에이전트로 활용하기에 부족했습니다. **한국어 중심이라면 클라우드 API를 주 모델로 사용하는 것을 권장합니다.** 다만 2026년 4월에 출시된 **Gemma 4(26B)**는 한국어 품질과 도구 호출 능력이 눈에 띄게 개선되었습니다. 스킬 활용, 외부 도구 호출 등의 에이전트 작업을 한국어로 수행하는 데 성공했습니다. RTX 4090(24GB)에서 32K 컨텍스트로 실행하면 VRAM을 초과하여 일부가 CPU로 오프로딩되는데, 이 경우 **속도는 클라우드의 가장 무거운 모델(Opus급) 정도, 품질은 가벼운 모델(Haiku급) 정도**로 체감됩니다. 비용 없이 데이터 주권을 확보할 수 있다는 점에서 시도해 볼 가치가 있습니다.*

Ollama 설치 및 모델 다운로드

```
# macOS/Linux
curl -fsSL https://ollama.com/install.sh | sh

# 모델 다운로드 (예시)
ollama pull hermes3
```

`ollama list`로 사용 가능한 모델을 확인하고, 용도에 맞는 모델을 선택합니다.

Hermes Agent에 Ollama 연동

`~/.hermes/config.yaml`에서 모델·프로바이더·엔드포인트를 지정합니다. v0.7.0부터 `config.yaml`이 엔드포인트 URL의 단일 출처입니다.

```
model:
  default: "<모델명>"        # 예: "hermes3", "qwen3:14b", "llama3.1:8b"
  provider: "custom"
  base_url: "http://<Ollama-서버-IP>:11434/v1"
```

`default`에는 `ollama list`에 나오는 모델명을 넣습니다.

- **base_url**: Ollama 서버 주소에 `/v1`을 붙입니다. 같은 머신이라면 `http://localhost:11434/v1` 입니다.
- **provider**: `"custom"`으로 지정하면 `base_url`로 요청이 갑니다.
- **API 키**: Ollama는 인증이 필요 없습니다. v2026.3.x(PR #3604) 이후 Hermes가 키를 못 찾으면 내부 더미값(`no-key-required`)을 자동으로 채워 SDK를 통과시키므로, `~/.hermes/.env`에 `OPENAI_API_KEY`를 따로 둘 필요가 없습니다. 키가 필요한 자체호스팅(vLLM 등)이라면 `~/.hermes/config.yaml`의 `custom_providers:` 리스트 항목에 `key_env: MY_KEY`를 지정하고 `~/.hermes/.env`에 `MY_KEY=…`를 두는 방식이 권장됩니다.

> ⚠️ ***v0.7.0 변경 사항**: 이전 버전에서는 `.env` 의 `OPENAI_BASE_URL` 환경변수로 엔드포인트를 지정할 수 있었지만, v0.7.0부터는 `config.yaml` 의 `model.base_url` 만 참조합니다. `.env` 의 `LLM_MODEL` 환경변수도 제거되었습니다. **시크릿(API 키)은 `.env` 에, 나머지 설정은 `config.yaml` 에** 두는 것이 원칙입니다.*

이 작업도 `hermes model` 로 인터랙티브하게 처리할 수 있습니다. 프로바이더 목록에서 **Custom endpoint**를 골라 `base_url` 을 입력하면, Hermes가 `config.yaml` 을 같은 형식으로 갱신합니다(2-3절의 `hermes model` 워크스루 참고).

LM Studio 정식 어댑터

v0.12부터 **LM Studio**가 정식 프로바이더로 들어왔습니다. Ollama와 달리 GUI에서 모델을 관리할 수 있고, `hermes doctor` 의 점검·라이브 `/models` 조회 같은 정식 통합을 그대로 이용할 수 있습니다.

```
model:
  default: "<LM Studio 모델 ID>"
  provider: "lmstudio"
  base_url: "http://127.0.0.1:1234/v1"   # 기본값과 같음
```

LM Studio 앱에서 서버를 켠 뒤 `hermes model` → **LM Studio**를 고르면 됩니다. `LM_API_KEY` 가 필요한 환경에서는 `.env` 에 같이 넣습니다.

컨텍스트 길이 설정

Hermes Agent는 모델의 컨텍스트 길이를 자동 감지하고, 사용량이 50%를 넘으면 자체 압축기로 오래된 대화를 요약합니다. 하지만 Ollama의 `num_ctx` 가 작으면 **Ollama 레벨에서 먼저 앞부분이 잘려나가** Hermes의 압축기가 개입할 기회 없이 이전 대화를 잊어버릴 수 있습니다. 최근 버전에서는 Ollama의 런타임 컨텍스트가 도구 사용에 필요한 최소치에 미달하면 대화 시작 시 경고를 표시하고 컨텍스트 확대를 안내합니다.

<fieldset>
<legend>Ollama의 기본 컨텍스트 길이</legend>

Ollama 0.20부터는 VRAM에 따라 기본 컨텍스트 길이가 자동 설정됩니다.

VRAM	기본 num_ctx
24GB 미만	4,096
24~48GB	32,768
48GB 이상	262,144
</fieldset>

명시적으로 변경하려면 환경변수로 설정합니다.

```
OLLAMA_CONTEXT_LENGTH=32768 ollama serve
```

`config.yaml`에서 Hermes 측 컨텍스트 길이를 명시할 수도 있습니다.

```
model:
  default: "<모델명>"
  provider: "custom"
  base_url: "http://<Ollama-서버-IP>:11434/v1"
  context_length: 32768
```

VRAM과 컨텍스트 길이의 트레이드오프

컨텍스트 길이를 늘리면 KV 캐시가 더 많은 VRAM을 사용합니다. RTX 4090(24GB)에서 Gemma 4 26B + 32K 컨텍스트를 실제 테스트한 결과, 모델과 KV 캐시를 합쳐 약 32GB가 필요했습니다. VRAM이 부족하면 Ollama가 일부를 CPU/RAM으로 자동 오프로딩하는데(`ollama ps`에서 `26%/74% CPU/GPU`로 표시), 시스템 RAM도 부족하면 OOM이 발생할 수 있습니다. 오프로딩이 되더라도 CPU 처리 비율만큼 추론 속도가 느려지므로, `watch -n 1 nvidia-smi`로 모니터링하면서 점진적으로 올리세요. Flash Attention(`OLLAMA_FLASH_ATTENTION=1`)을 켜면 VRAM 효율이 개선됩니다. KV 캐시 양자화(`OLLAMA_KV_CACHE_TYPE=q8_0`)를 적용하면 KV 캐시의 VRAM 사용량이 절반으로 줄어 오프로딩 비율을 낮출 수 있습니다. `q4_0`은 1/4로 줄지만 품질 손실이 있을 수 있습니다.

LAN 환경에서의 연동

LLM 서버가 LAN 내 다른 머신에 있는 경우, Ollama 서버가 외부 접속을 허용해야 합니다.

```
# Ollama가 모든 인터페이스에서 수신하도록 설정
OLLAMA_HOST=0.0.0.0 ollama serve
```

또는 systemd 서비스 파일에서:

```
[Service]
Environment="OLLAMA_HOST=0.0.0.0"
```

로컬과 클라우드 혼합 전략

일상적인 작업은 LAN 서버의 로컬 모델로 처리하고, 복잡한 판단이 필요한 작업에만 클라우드 API를 사용하는 혼합 전략을 권장합니다. `config.yaml` 에서 `smart_model_routing` 을 활성화하면 작업 복잡도에 따라 자동으로 모델을 전환합니다(3-3절 참고). 보조 모델 슬롯도 마찬가지로 vision·web_extract만 로컬에 두고 나머지는 클라우드 기본값으로 둘 수 있습니다(2-3절「보조 모델 설정」참고).

연동 확인

```
# 로컬 모델(Ollama) 직접 지정
hermes chat -q "Hello, what model are you?" --provider custom -m hermes3
```

문제가 있다면 `hermes doctor` 로 진단합니다.

실제 테스트 비교

다음 비교는 macOS Intel Mac에서 LAN 내 Ollama 서버(RTX 4090 한 장을 장착한 게임용 데스크톱 PC)로 연결하여 진행했습니다 (2026년 3월 기준, Hermes Agent v0.4.0).

작업	Llama 3.1 8B	Qwen 2.5 14B	Hermes 3 (8B)	Qwen3 14B
파일 생성	❌ 가짜 경로	✅ 성공	✅ 성공	✅ 성공
코드 실행 (execute_code)	미테스트	❌ `\n` 버그 10회 실패	✅ 성공	✅ 성공
코드 실행 (터미널)	미테스트	✅ 우회 성공	✅ 성공	✅ 성공
웹 검색	미테스트	❌ think만 생성	미테스트	미테스트
메모리 저장/회상	미테스트	✅ 성공	✅ 성공	✅ 성공
시스템 정보 조회	미테스트	△ Linux 명령 시도	△ 디스크 정확, 메모리 일부 실패	△ 디스크 정확, 메모리 미테스트

핵심 발견

Hermes 3는 8B임에도 에이전트 작업에서 뛰어납니다. 같은 8B급인 Llama 3.1이 가짜 경로를 사용하여 실패한 것과 대조적으로, Hermes 3는 파일 생성, 코드 실행, 메모리 3개 핵심 테스트를 모두 통과했습니다. NousResearch가 도구 호출 패턴에 맞춰 파인튜닝한 효과가 분명히 있습니다. 다만 macOS 시스템 명령에서는 존재하지 않는 sysctl 키를 시도하는 등 완벽하지는 않습니다.

Qwen3 14B는 Qwen 2.5 14B의 약점을 극복했습니다. Qwen 2.5에서 실패했던 `execute_code` 도구가 Qwen3에서는 정상 작동하며, macOS에서 올바른 명령(`df -h`)을 사용합니다. 한국어 응답 품질도 향상되었습니다.

8B 범용 모델은 에이전트용으로 부족합니다. Llama 3.1 8B는 도구 호출 시 실제 파일 시스템 경로 대신 플레이스홀더(`/path/to/`, `/home/username/`)를 넣는 경향이 있어 ReAct 루프가 제대로 작동하지 않습니다. 다만 Hermes 3처럼 에이전트용으로 파인튜닝된 8B 모델은 예외입니다.

코딩 벤치마크 점수와 에이전트 호환성은 다른 문제입니다. 코딩 벤치마크에서 높은 점수를 받은 모델이라도 도구 호출(function calling)을 네이티브로 지원하지 않으면 Hermes Agent에서 사용할 수 없습니다. 예를 들어 DeepSeek-Coder-V2 Lite(16B)는 코드 생성 능력이 뛰어나지만, 도구 호출 훈련이 되어 있지 않아 Hermes Agent의 에이전트 루프가 작동하지 않습니다. 모델을 고를 때 "코딩을 잘하는가"보다 "도구를 호출할 수 있는가"를 먼저 확인해야 합니다.

VRAM별 모델 추천

상황	추천 모델	이유
VRAM 8GB 이하	**Hermes 3**	에이전트 특화 파인튜닝, 6GB로 전 기능 사용
VRAM 12GB	**Qwen3 14B**	도구 호출 + 한국어 모두 우수
VRAM 24GB 이상	**Qwen3 32B**	로컬 모델 중 최상급
스마트 라우팅 조합	Qwen3 14B (주) + Hermes 3 (보조)	비용 대비 최적

에이전트용 유망 로컬 모델 (2026년 기준)

Hermes Agent와 같은 도구 호출 기반 에이전트에서는 모델 크기나 코딩 벤치마크 점수보다 **도구 호출(function calling) 네이티브 지원 여부**가 가장 중요한 선택 기준입니다. 도구 호출을 지원하지 않는 모델은 응답을 일반 텍스트로만 생성하기 때문에, 에이전트가 파일을 만들거나 명령을 실행하는 등의 행동을 할 수 없습니다. 2026년 3월 기준으로 Ollama에서 사용할 수 있는 유망한 모델을 정리합니다.

모델	Ollama 명령	VRAM	도구 호출	한국어	비고
Hermes 3	`ollama pull hermes3`	~6GB	✅ 테스트 통과	보통	NousResearch 자체 모델, 8B인데도 전 기능 정상
Qwen3-14B	`ollama pull qwen3:14b`	~10GB	✅ 테스트 통과	우수	Qwen 2.5 14B 대비 크게 향상
Qwen3.5-9B	`ollama pull qwen3.5:9b`	~6GB	좋음	우수	2026년 2월 출시, 네이티브 도구 호출, 멀티모달
Qwen3-32B	`ollama pull qwen3:32b`	~20GB	최상	우수	24GB VRAM 이상 시 최적 선택
GPT-OSS	`ollama pull gpt-oss`	~15GB	매우 좋음	보통	OpenAI 오픈소스, 안정적 도구 호출

벤치마크 점수와 실제 에이전트 작업 성능은 다를 수 있습니다. 특히 한국어 환경에서는 어떤 로컬 모델이든 클라우드 API 대비 격차가 큽니다. 로컬 모델은 영어 중심의 단순 작업에 활용하고, 한국어나 복잡한 지시는 클라우드 API와 병행하는 것이 현실적입니다.

음성 모드 쪽에서는 v0.12부터 **Piper**가 네이티브 로컬 TTS로 추가됐습니다(3-5절). 외부 API 키 없이 한국어 음성 응답을 띄울 때 가벼운 선택지입니다.

로컬 모델을 판정 모델로 활용하기

10-4절의 `/goal` 기능은 매 턴마다 판정 에이전트가 목표 달성 여부를 확인합니다. 이 판정 에이전트(`auxiliary.goal_judge`)에 로컬 모델을 쓰면 판정 비용을 완전히 없앨 수 있습니다.

판정 작업은 비교적 단순합니다. 목표 텍스트(최대 2,000자)와 에이전트의 마지막 응답(최대 4,000자)을 받아 `{"done": true/false, "reason": "..."}` 형태의 JSON 한 줄을 반환하면 됩니다. Hermes의 파서도 관대해서, 마크다운 코드 펜스로 감싼 JSON이나 `"done": "true"`(문자열)도 허용합니다. 이 정도의 구조화 출력이라면 로컬 모델로도 충분히 처리할 수 있습니다.

```
# config.yaml
auxiliary:
  goal_judge:
    provider: custom
    base_url: "http://localhost:11434/v1"
    model: gemma4:26b
```

CLI로도 설정할 수 있습니다:

```
hermes config set auxiliary.goal_judge.provider custom
hermes config set auxiliary.goal_judge.base_url "http://localhost:11434/v1"
hermes config set auxiliary.goal_judge.model gemma4:26b
```

후보 모델: Gemma 4 26B는 네이티브 function-calling 토큰 6개를 내장하고 있어 JSON 출력 신뢰도가 높고, RTX 4090(24GB)에 적재할 수 있습니다. 더 가벼운 모델(Gemma 4 12B, Qwen3 14B 등)도 이 정도의 단순 판정에는 사용할 수 있지만, 모델이 작을수록 JSON 대신 산문을 반환할 가능성이 높아집니다.

주의할 점:

- **타임아웃**: auxiliary 호출의 기본 타임아웃은 30초입니다. 로컬 모델은 이보다 느릴 수 있으므로 `auxiliary.goal_judge.timeout` 을 넉넉히 설정하는 것을 권장합니다.
- **파싱 실패 자동정지**: 판정 모델이 유효한 JSON을 반환하지 못하면 3회 연속 실패 시 목표가 자동 일시정지됩니다. 이 메시지가 반복되면 더 큰 모델로 바꾸거나 클라우드 판정 모델로 전환하는 것을 권장합니다.

외부 메모리 백엔드(Honcho)에 로컬 LLM 연결하기

5-3절에서 설명한 Honcho 같은 외부 메모리 백엔드는 서버 자체가 LLM을 호출합니다. 관찰 추출, 변증법적 추론, 요약, 임베딩을 모두 서버 측에서 처리하므로, Hermes의 LLM과는 별도의 모델 설정이 필요합니다.

기본값은 OpenAI API(gpt-5.4-mini + text-embedding-3-small)이지만, Ollama를 연결하면 외부 API 의존을 완전히 제거할 수 있습니다.

Honcho의 .env 설정 (Ollama 연결)

Honcho 서버의 `.env` 파일에서 각 모듈의 모델과 엔드포인트를 지정합니다.

```
# LLM (관찰 추출·변증법·요약·Dream)
DERIVER_MODEL_CONFIG__TRANSPORT=openai
DERIVER_MODEL_CONFIG__MODEL=qwen3:14b
DERIVER_MODEL_CONFIG__OVERRIDES__BASE_URL=http://host.docker.internal:11434/v1

DIALECTIC_MODEL_CONFIG__TRANSPORT=openai
DIALECTIC_MODEL_CONFIG__MODEL=qwen3:14b
DIALECTIC_MODEL_CONFIG__OVERRIDES__BASE_URL=http://host.docker.internal:11434/v1

# 임베딩
EMBEDDING_MODEL_CONFIG__TRANSPORT=openai
EMBEDDING_MODEL_CONFIG__MODEL=rjmalagon/gte-qwen2-1.5b-instruct-embed-f16
EMBEDDING_MODEL_CONFIG__OVERRIDES__BASE_URL=http://host.docker.internal:11434/v1

# Ollama는 API 키 불필요
LLM_OPENAI_API_KEY=no-key-required
```

> 💡 ***transport가 openai인 이유**: Ollama는 OpenAI 호환 API를 제공하므로 `transport=openai` 로 지정하고 `BASE_URL` 만 Ollama 주소로 바꾸면 됩니다.*

`host.docker.internal` 은 Docker 컨테이너 안에서 호스트 머신의 Ollama에 접근하는 주소입니다. Ollama가 다른 머신에 있다면 해당 IP로 바꾸면 됩니다.

모델 선택 시 주의할 점

LLM 모델: Honcho는 관찰 추출 결과를 JSON으로 파싱합니다. **도구 호출(function calling)을 지원하고 구조화 출력이 안정적인 모델**이 필요합니다. qwen3:14b는 thinking 토큰이 별도 필드로 분리되어 JSON 출력을 오염시키지 않으므로 적합합니다.

임베딩 모델: Honcho의 pgvector 기본 차원은 **1536**입니다. 1536차원을 네이티브로 출력하는 모델을 선택하면 데이터베이스 스키마를 수정할 필요가 없습니다. `gte-Qwen2-1.5B-instruct` (1536차원)가 이 조건에 맞습니다.

> ⚠️ ***비-1536 임베딩 모델 사용 시**: bge-m3(1024차원) 같은 모델도 코드상 지원되지만, pgvector 컬럼을 `ALTER TABLE` 로 변경해야 하고, 실전에서 예상치 못한 문제가 발생할 수 있습니다.*

.env 변경 후 반영

```
# restart는 .env 변경을 반영하지 않음 — 반드시 up -d 사용
docker compose up -d
```

에이전트의 토큰 오버헤드

에이전트는 매 턴마다 시스템 프롬프트, 도구 정의, 스킬 목록, 메모리 등을 함께 전송합니다. 같은 질문이라도 Ollama에 직접 보내는 것과 Hermes Agent를 경유하는 것은 토큰 소비가 크게 다릅니다.

실험 환경: Gemma 4 26B (Ollama), 도구 29개, 스킬 116개 등록 상태

사용한 질문 (4턴 연속 대화):

1. "대한민국의 수도는?"
2. "그 도시의 인구는 대략 몇 명이야?"
3. "서울에서 가장 유명한 관광지 3곳을 알려줘"
4. "그중 경복궁의 입장료와 운영시간을 알려줘"

	Ollama 직접	Hermes 경유
1턴	~120	~14,200
4턴 누적	~2,100	~87,100

1턴 기준 약 14,000토큰의 차이가 에이전트의 고정 오버헤드입니다. 커뮤니티에서도 비슷한 수치가 보고되고 있습니다. 한 사용자가 토큰 분석 대시보드를 만들어 조사한 결과, 매 API 호출의 약 73%가 고정 오버헤드이며 도구 정의만으로 거의 절반을 차지한다고 합니다. CLI에서 6~8K, 메시징 플랫폼(Telegram 등)에서 15~20K 수준으로, 설정에 따라 차이가 있습니다.

비용에 미치는 영향

클라우드 API를 사용하면 이 오버헤드가 토큰 단가에 직접 곱해집니다. 다만 Claude API는 프롬프트 캐싱을 지원하므로, 반복되는 시스템 프롬프트를 캐시에서 읽으면 실질 비용이 크게 줄어듭니다.

로컬 모델은 토큰당 과금이 없으므로 비용에는 영향이 없습니다. 대신 컨텍스트 윈도우를 더 빨리 소진하고, 입력이 길어지는 만큼 추론 속도가 느려집니다.

오버헤드를 줄이려면 사용하지 않는 도구를 비활성화하거나, 메모리 크기를 적절히 제한하는 방법이 있습니다. 자세한 내용은 5장과 7장 스킬 시스템을 참고하세요.

컨텍스트 한계와 워크플로 선택

오버헤드는 로컬 모델의 컨텍스트 윈도우를 압박합니다. RTX 4090(24GB)에서 Ollama가 자동 설정하는 num_ctx는 32,768토큰인데, 매 턴 ~14,000토큰이 오버헤드로 사용되므로 실제 대화에 쓸 수 있는 공간은 약 18,000토큰입니다.

단답형 질문이나 간단한 도구 호출은 문제없지만, 도구 호출이 많은 다단계 워크플로에서는 컨텍스트가 빠르게 소진됩니다. 실제로 Gemma 4 26B + 32K 컨텍스트 환경에서 유튜브 자막 추출 → 요약 → 블로그 작성 워크플로를 수행한 결과, 64개 메시지에 걸쳐 누적 입력 토큰이 582K에 달했습니다(매 턴 재전송 합산). 대화 후반에 모델이 초반에 받은 URL을 기억하지 못하는 현상이 관찰되었습니다. 이런 도구 호출이 많은 다단계 워크플로는 128K 이상의 컨텍스트를 제공하는 클라우드 API가 적합합니다. 또는 DGX Spark처럼 128GB 이상의 통합 메모리를 갖춘 로컬 시스템이라면 긴 컨텍스트를 로컬에서도 확보할 수 있습니다.

12장에서 설명하듯 LLM API는 무상태이므로, 오버헤드(~14,000토큰)는 한 번이 아니라 **매 턴마다** 다시 전송됩니다. 여기에 이전 대화 이력이 턴마다 누적되므로, N번째 턴의 입력 크기는 대략 `14,000 + 1~N턴 대화 이력` 이 됩니다. 이 합이 num_ctx에 근접하면 컨텍스트 한계입니다.

한편, 단답형 질문처럼 도구 호출이나 메모리가 필요 없는 작업이라면 에이전트를 거칠 이유가 없습니다. `ollama run` 이나 API를 직접 호출하는 것이 훨씬 효율적입니다. 로컬 모델 + 에이전트의 적합 영역은 **도구 호출이나 메모리가 필요하면서, 컨텍스트 안에 들어오는 길이의 작업**입니다. 이 범위를 넘어서는 긴 워크플로는 클라우드 API가 현실적입니다.